

Universidad ORT Uruguay

Facultad de Ingeniería

Herramientas de software para Big Data

Obligatorio

Documentación

Integrantes:

Juan Diego Fagnoni - 288818

Francisco Baraibar - 285465

Docentes: Eduardo Garcia, Alexis Arriola

9 de julio, 2025

Link al dataset:

<https://www.kaggle.com/datasets/aryashah2k/beginners-sports-analytics-nfl-dataset>

Índice

Búsqueda y validación del Dataset.....	3
Dominio y estructura del Dataset.....	3
Preparación de ingesta.....	4
Creación de regiones.....	5
Propósito de cada región.....	5
Ingesta de tablas.....	6
Componentes del flujo.....	7
Configuración de los procesadores.....	8
Análisis y refinamiento de datos.....	10
Modelado de los datos.....	11
Justificación de selección del modelo normalizado.....	11
Representación del modelo.....	13
Tablas HIVE.....	14
Organización previa al creado de tablas.....	14
Movimiento de datos crudos iniciales.....	14
Verificación de archivos refinados.....	14
Creación de tablas externas en HIVE.....	15
Respuesta a las preguntas con HIVE y Spark.....	19
Guardado de respuestas.....	19
Visualización de Resultados en Notebook.....	20
Pregunta 2: ¿Cuáles fueron los 5 equipos que generaron la mayor cantidad de jugadas ofensivas con impacto positivo en la probabilidad de anotar en cada semana?.....	21
Pregunta 3: ¿Qué jugadores alcanzaron la mayor velocidad promedio por posición ofensiva en jugadas de pase exitoso?.....	21
Pregunta 5: ¿Qué universidades formaron a los 10 jugadores que más distancia recorrieron durante toda la temporada?.....	22
Visualización de resultados en Superset.....	23
Creación de nuevas tablas en Hive.....	23
Ingresamos a Superset.....	24
Creación de Charts.....	24
Pregunta 1: ¿Cuáles son los estados con mayor cantidad de partidos disputados?.....	25
Pregunta 4: ¿Qué equipos permitieron la menor cantidad de yardas por jugada en contra a lo largo de la temporada?.....	26
Pregunta 5: ¿Qué universidades formaron a los 10 jugadores que más distancia recorrieron durante toda la temporada?.....	27
Dashboard Final.....	28
Parte 2 - Arquitectura Alternativa para un Data Lake.....	29
Stack tecnológico del nuevo data lake.....	29
Flujo del proceso.....	30
Comparación entre arquitecturas.....	31
Diferencias clave:.....	32
Visualización del flujo de datos.....	33
Organización por zonas en el data lake.....	34

Búsqueda y validación del Dataset

Para comenzar con el obligatorio, lo primero que hicimos fue una búsqueda en Kaggle con el objetivo de encontrar un dataset que cumpla con los requisitos esperados para trabajar. Es decir, que tuviera al menos 4 tablas y que al menos una de ellas tuviera más de 10.000 registros.

Luego de buscar varias opciones y hablar con los docentes, determinamos que el dataset que íbamos a usar era **Beginners Sports Analytics NFL Dataset**.

Para poder darle uso, creamos la tabla teams.csv, que contiene todos los equipos de la NFL con su abreviatura, estado perteneciente y su id.

Dominio y estructura del Dataset

El dataset que elegimos contiene información detallada sobre partidos, jugadores, equipos y jugadas de la NFL.

Está orientado al análisis deportivo y su dominio gira en torno al desarrollo de los partidos de la temporada.

La información está organizada en torno a cinco dimensiones principales:

Games: cada registro representa un partido específico, con información sobre fecha, equipos involucrados, estadio, condiciones del clima, entre otros.

Players: se detallan atributos individuales como nombre, posición, equipo, universidad de origen, altura, peso y número de identificación.

Teams: contiene los datos institucionales de cada franquicia (id, abreviatura, estado).

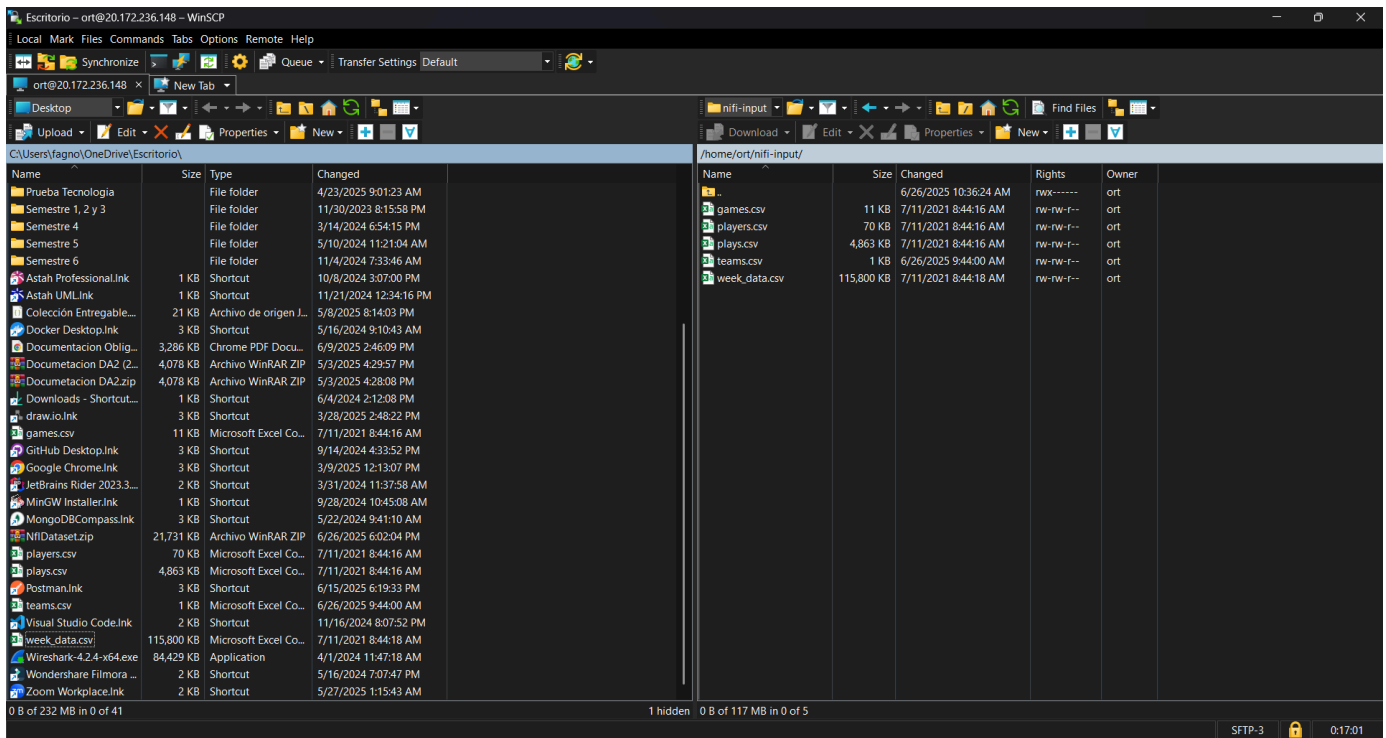
Plays: describe cada jugada del partido, incluyendo tipo de jugada, down, y situación de campo.

Week_Data: captura métricas detalladas por jugada y jugador, como distancia recorrida, velocidad, aceleración, posición en el campo, etc. Esta tabla es la mas granular y permite que se hagan análisis físicos o de rendimiento de los jugadores durante distintas jugadas de cada partido.

Esta información permite plantear preguntas sobre distintas perspectivas, combinando aspectos físicos, estratégicos y estadísticos.

Preparación de ingesta

Una vez definido el dataset a utilizar, creamos en nuestra máquina virtual la carpeta /nifi-input. El propósito de esta carpeta es contener los datos en un lugar accesible para NiFi, desde donde se pueda leer los archivos a procesar.



Para cargar los datos en /nifi-input utilizamos la herramienta WinSCP, que nos permitió cargar los cinco archivos .csv desde nuestro entorno local a la carpeta contenida en la máquina virtual.

En la imagen se muestra el uso de la herramienta para cargar el .zip desde el escritorio a la carpeta.

Una vez hecho esto, chequeamos en consola que se haya cargado correctamente.

```
[ort@taller-bigdata-vm ~]$ cd nifi-input
[ort@taller-bigdata-vm nifi-input]$ ls
games.csv  players.csv  plays.csv  teams.csv  week_data.csv
```

Creación de regiones

En esta etapa creamos cuatro carpetas que definen las distintas zonas del data lake de nuestro HDFS. El objetivo de esta estructura es mantener un flujo de datos ordenado y reproducible a lo largo de todo su ciclo de vida, desde la ingesta hasta su análisis final. A continuación se muestra cómo creamos las carpetas:

```
ort@taller-bigdata-vm ~]$ hdfs dfs -mkdir -p /user/bigdata/zones/Ind
ort@taller-bigdata-vm ~]$ hdfs dfs -mkdir -p /user/bigdata/zones/raw
ort@taller-bigdata-vm ~]$ hdfs dfs -mkdir -p /user/bigdata/zones/refined
ort@taller-bigdata-vm ~]$ hdfs dfs -mkdir -p /user/bigdata/zones/analytics
```

Luego de crearlas, chequeamos que se hayan creado correctamente:

```
ort@taller-bigdata-vm ~]$ hdfs dfs -ls /user/bigdata/zones
Found 4 items
drwxr-xr-x - ort supergroup          0 2025-06-26 13:05 /user/bigdata/zones/analytics
drwxr-xr-x - ort supergroup          0 2025-06-26 13:05 /user/bigdata/zones/Ind
drwxr-xr-x - ort supergroup          0 2025-06-26 13:05 /user/bigdata/zones/raw
drwxr-xr-x - ort supergroup          0 2025-06-26 13:05 /user/bigdata/zones/refined
```

Propósito de cada región

Landing (Ind): en esta zona se almacenan los archivos que llegan a través del flujo de NiFi. Al llegar se realiza un primer análisis exploratorio y de refinamiento usando Spark. En este proceso eliminamos duplicados, valores nulos, realizamos casting de columnas al tipo de datos más adecuado para el análisis y eliminamos aquellas columnas que consideramos que no aportaban información valiosa para obtener las respuestas de las preguntas seleccionadas.

Raw (raw): esta zona actúa como respaldo de los archivos originales que llegan inicialmente a landing. La idea es que, una vez que los datos fueron tratados y almacenados en su subcarpeta correspondiente en refined, los datos crudos se muevan a esta carpeta para conservar una copia inalterada. Esto permite que, si en un futuro se agregan nuevas tablas o se quiere reprocesar desde cero, se pueda partir desde una base limpia sin interferencia de datos que ya fueron tratados.

Refined(refined): acá guardamos las versiones limpias de los datos, después del análisis exploratorio y refinado. En esta etapa ya se corrigieron los valores nulos, se eliminaron duplicados, se definieron los tipos de datos y analizaron las posibles claves primarias.

Cada tabla refinada se almacena en una subcarpeta correspondiente dentro de la carpeta refined. Estas ubicaciones son las que usan las tablas externas de Hive para poder hacer las consultas analíticas.

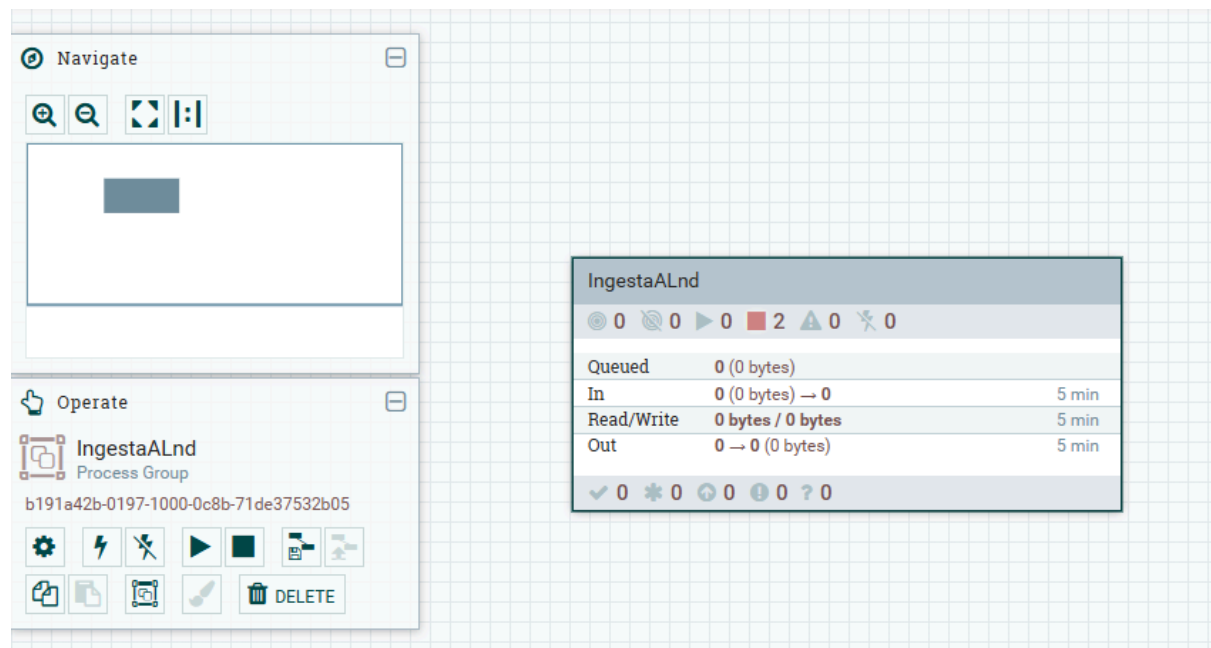
Analytics(analytics): esta zona contiene los resultados de las consultas realizadas sobre el modelo de datos. Se almacenan tanto los archivos con resultados como las tablas HIVE asociadas, las cuales alimentan notebooks para visualización y análisis.

Ingesta de tablas

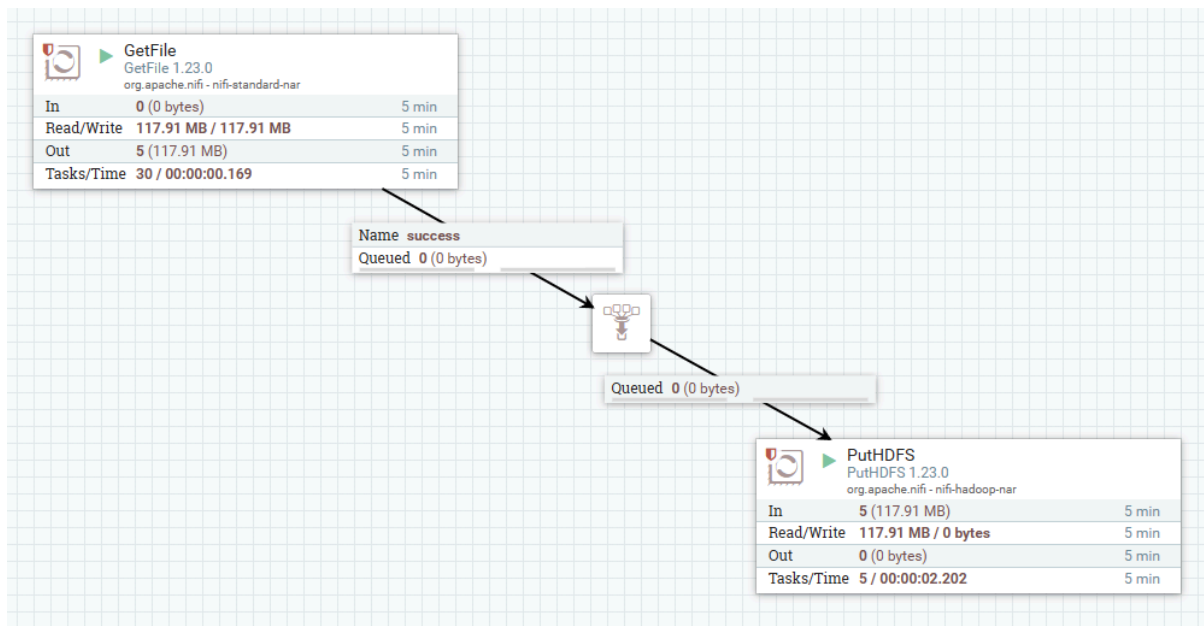
En esta sección movimos los archivos desde la carpeta /nifi-input (ubicada en nuestra máquina virtual), hacia la zona landing en HDFS, diseñando un flujo en NiFi.

Para organizar el flujo de forma clara y reutilizable, creamos un Process Group llamado IngestaALnd. Esto nos permite encapsular el proceso de ingesta, facilitando tanto la comprensión como la posibilidad de reutilizar o modificar el flujo en el futuro sin afectar otros componentes

En la primera imagen vemos el Process Grupo IngestaALnd en NiFi.



Al hacer doble click en el Process group accedemos al mismo, pudiendo visualizar el flujo diseñado.



Una vez hecho esto, le damos start al process group corriendo todos los componentes. Como vemos en la imagen los mismos se mueven como se esperaba de origen a destino. El procesador GetFile marca cinco archivos Out, mientras que el procesador PutHDFS marca cinco archivos In.

Para verificar que se hayan movido correctamente, aplicamos el siguiente comando en consola:

```
[ort@taller-bigdata-vm ~]$ hdfs dfs -ls /user/bigdata/zones/1nd
Found 5 items
-rw-r--r-- 1 ort supergroup 10451 2025-06-27 14:15 /user/bigdata/zones/1nd/games.csv
-rw-r--r-- 1 ort supergroup 71358 2025-06-27 14:15 /user/bigdata/zones/1nd/players.csv
-rw-r--r-- 1 ort supergroup 4979613 2025-06-27 14:15 /user/bigdata/zones/1nd/plays.csv
-rw-r--r-- 1 ort supergroup 577 2025-06-27 14:15 /user/bigdata/zones/1nd/teams.csv
-rw-r--r-- 1 ort supergroup 118578223 2025-06-27 14:15 /user/bigdata/zones/1nd/week_data.csv
```

Componentes del flujo

- **GetFile:** es el punto de partida del flujo. Su función es leer archivos locales desde una ruta específica del sistema de archivos de la máquina virtual. En nuestro caso, se configuró para que apunte a la carpeta /nifi-input, donde teníamos los 5 archivos .csv. Una vez leído el contenido, se envía a través de la relación success al siguiente paso del flujo.

- **Funnel:** lo incluimos como buena práctica para organizar y centralizar el flujo. Si bien en este caso incluirlo no era totalmente necesario, ya que los datos van directamente desde GetFile hacia PutHDFS, nos pareció bien hacerlo para dar una estructura escalable y principalmente para aplicar las herramientas y componentes vistos en el curso.

-PutHDFS: este procesador escribe los archivos leídos en el HDFS, en la zona landing (Ind). Cada archivo se guarda sin modificaciones actuando como respaldo del archivo original.

Por último, nos parece correcto mencionar que podríamos haber utilizado el procesador UpdateAttribute ya que fue utilizado en el práctico de NiFi del curso y se vio en clase. Este permite modificar o agregar atributos a los archivos, como por ejemplo renombrarlos o asignar etiquetas personalizadas.

Sin embargo, en nuestro caso decidimos no incluirlo ya que los archivos ya contaban con nombres claros y no consideramos necesario hacer modificaciones adicionales.

Configuración de los procesadores

En esta sección evidenciamos a través de imágenes como fueron configurados los procesadores para que el flujo tenga el comportamiento esperado.

Configuración GetFile:

Configure Processor | GetFile 1.23.0

Stopped

SETTINGS | SCHEDULING | **PROPERTIES** | RELATIONSHIPS | COMMENTS

Required field  

Property	Value
Input Directory	 /home/ort/nifi-input
File Filter	 *.*.csv
Path Filter	 No value set
Batch Size	 10
Keep Source File	 false
Recurse Subdirectories	 true
Polling Interval	 0 sec
Ignore Hidden Files	 true
Minimum File Age	 0 sec
Maximum File Age	 No value set
Minimum File Size	 0 B
Maximum File Size	 No value set

CANCELAPPLY

Para adaptar GetFile a nuestro caso, hicimos dos configuraciones:

- 1) **Input Directory:** /home/ort/nifi-input, esta es la carpeta que configuramos como origen de los datos. Ahí estaban los cinco .csv y desde esa ubicación NiFi los toma para comenzar el procesamiento
- 2) **File Filter:** *.*\, definimos este filter para asegurar que el procesador capture todos los archivos con extensión .csv. Se hizo más bien como buena práctica ya que todos los archivos que tenía la carpeta eran .csv.

El resto de las configuraciones del procesador se mantuvieron con sus valores por defecto.

Configuración PutHDFS:

Configure Processor | PutHDFS 1.23.0

Stopped

SETTINGS SCHEDULING PROPERTIES RELATIONSHIPS COMMENTS

Required field

Property	Value
Hadoop Configuration Resources	/home/ort/hadoop/etc/hadoop/core-site.xml,/home/ort...
Kerberos Credentials Service	No value set
Kerberos User Service	No value set
Kerberos Principal	No value set
Kerberos Keytab	No value set
Kerberos Password	No value set
Kerberos Relogin Period	4 hours
Additional Classpath Resources	No value set
Directory	/user/bigdata/zones/Ind/
Conflict Resolution Strategy	replace
Writing Strategy	Write and rename
Block Size	No value set

CANCEL APPLY

Para que el PutHDFS actúe como esperábamos, modificamos las siguientes propiedades:

- 1) **Hadoop Configuration Resources con:**
/home/ort/hadoop/etc/hadoop/core-site.xml,/home/ort/hadoop/etc/hadoop/hdfs-site.xml, esto permite que NiFi se conecte al clúster HDFS utilizando los archivos de configuración propios de Hadoop. Indicamos la ruta a los archivos core-site.xml y hdfs-site.xml ubicados en nuestra máquina virtual, los cuales contienen la

información necesaria sobre el NameNode, puertos y política de acceso del sistema distribuido.

- 2) **Directory:** /user/bigdata/zones/lnd/, esta es la ruta dentro del HDFS donde almacenamos los archivos, es a donde llegan los datos crudos.
- 3) **Conflict Resolution Strategy:** replace, asegura que en caso de que un archivo con el mismo nombre ya exista en la carpeta destino, sea reemplazado automáticamente. Esto garantiza que siempre se mantenga la versión más reciente de los datos sin generar duplicados.

Las demás configuraciones se mantuvieron como se encontraban por defecto.

Análisis y refinamiento de datos

Una vez que los datos llegaron a landing a través del flujo, lo que hicimos fue crear un notebook en Jupyter, llamado analisisYRefinamiento_De_Datos.

En este notebook hicimos un análisis exploratorio y refinamiento de los datos utilizando Spark.

El propósito de este paso fue:

- Entender la estructura y el contenido de cada una de las tablas.
- Verificar la calidad de los datos antes de su análisis.
- Establecer las claves primarias esperadas para cada tabla.

Para esto hicimos una revisión de los tipos de datos y de los valores nulos y duplicados, además de la validación de unicidad de identificadores. También describimos el significado de cada columna de las tablas dentro del dominio de la NFL, pudiendo entender más las posibles relaciones entre las tablas.

Hecha la limpieza y validación de los datos, los archivos refinados fueron guardados en HDFS en la ubicación /user/bigdata/zones/refined/

Cada subcarpeta dentro de refined contiene cada una de las tablas limpias.

Esta versión refinada de los datos es la que utilizamos para el modelado y la creación de las tablas externas en Hive.

Aclaración: dentro del notebook entregado se encuentran distintos markdown que dan una explicación más detallada del análisis y refinamiento de cada tabla.

En esta imagen vemos la creación de las subcarpetas y el chequeo en consola de que las tablas refinadas se guardaron en la ubicación esperada.

```
ort@taller-bigdata-vm ~]$ hdfs dfs -ls /user/bigdata/zones/refined
Found 5 items
drwxr-xr-x - ort supergroup      0 2025-06-27 20:17 /user/bigdata/zones/refined/games
drwxr-xr-x - ort supergroup      0 2025-06-27 20:01 /user/bigdata/zones/refined/players
drwxr-xr-x - ort supergroup      0 2025-06-27 20:40 /user/bigdata/zones/refined/plays
drwxr-xr-x - ort supergroup      0 2025-06-27 14:46 /user/bigdata/zones/refined/teams
drwxr-xr-x - ort supergroup      0 2025-06-27 17:52 /user/bigdata/zones/refined/week_data
ort@taller-bigdata-vm ~]$ hdfs dfs -ls /user/bigdata/zones/refined/teams
Found 2 items
-rw-r--r-- 1 ort supergroup      0 2025-06-27 14:46 /user/bigdata/zones/refined/teams/_SUCCESS
-rw-r--r-- 1 ort supergroup    511 2025-06-27 14:46 /user/bigdata/zones/refined/teams/part-00000-97e37fae-0c06-4a82-9440-24896e8e6ad1-c000.csv
ort@taller-bigdata-vm ~]$ hdfs dfs -ls /user/bigdata/zones/refined/plays
Found 2 items
-rw-r--r-- 1 ort supergroup      0 2025-06-27 20:40 /user/bigdata/zones/refined/plays/_SUCCESS
-rw-r--r-- 1 ort supergroup 1341865 2025-06-27 20:40 /user/bigdata/zones/refined/plays/part-00000-c9dc69f5-b886-477d-b04c-e952fd276454-c000.csv
ort@taller-bigdata-vm ~]$ hdfs dfs -ls /user/bigdata/zones/refined/players
Found 2 items
-rw-r--r-- 1 ort supergroup      0 2025-06-27 20:01 /user/bigdata/zones/refined/players/_SUCCESS
-rw-r--r-- 1 ort supergroup    57908 2025-06-27 20:01 /user/bigdata/zones/refined/players/part-00000-8c4d94ef-00e4-406a-9a59-27c75a64c94b-c000.csv
ort@taller-bigdata-vm ~]$ hdfs dfs -ls /user/bigdata/zones/refined/games
ls: '/user/bigdata/zones/refined/games': No such file or directory
ort@taller-bigdata-vm ~]$ hdfs dfs -ls /user/bigdata/zones/refined/games
Found 2 items
-rw-r--r-- 1 ort supergroup      0 2025-06-28 17:17 /user/bigdata/zones/refined/games/_SUCCESS
-rw-r--r-- 1 ort supergroup    10385 2025-06-28 17:17 /user/bigdata/zones/refined/games/part-00000-e43084b8-da13-469e-9849-a6b7bce401db-c000.csv
ort@taller-bigdata-vm ~]$ hdfs dfs -ls /user/bigdata/zones/refined/week_data
Found 3 items
-rw-r--r-- 1 ort supergroup      0 2025-06-28 16:56 /user/bigdata/zones/refined/week_data/_SUCCESS
-rw-r--r-- 1 ort supergroup 42798804 2025-06-28 16:56 /user/bigdata/zones/refined/week_data/part-00000-4da0285a-d677-4ee5-8f49-0ee5bdf1b3a7-c000.csv
-rw-r--r-- 1 ort supergroup 42849166 2025-06-28 16:56 /user/bigdata/zones/refined/week_data/part-00001-4da0285a-d677-4ee5-8f49-0ee5bdf1b3a7-c000.csv
```

Modelado de los datos

Una vez que hicimos el análisis exploratorio y limpieza de los datos, elegimos el modelado para estructurar las relaciones entre nuestras tablas, permitiendo luego su análisis mediante consultas.

En nuestro caso optamos por un **modelo normalizado**

Justificación de selección del modelo normalizado

Motivos de la elección:

- Las tablas originales del dataset elegido ya presentan una estructura relacional clara y bien definida que se basa en identificadores únicos como: nflId, gameId, playId, teamAbbreviation y más.
- No es necesario desnormalizar ni crear tablas adicionales para realizar consultas, los datos están organizados de manera coherente desde el origen.

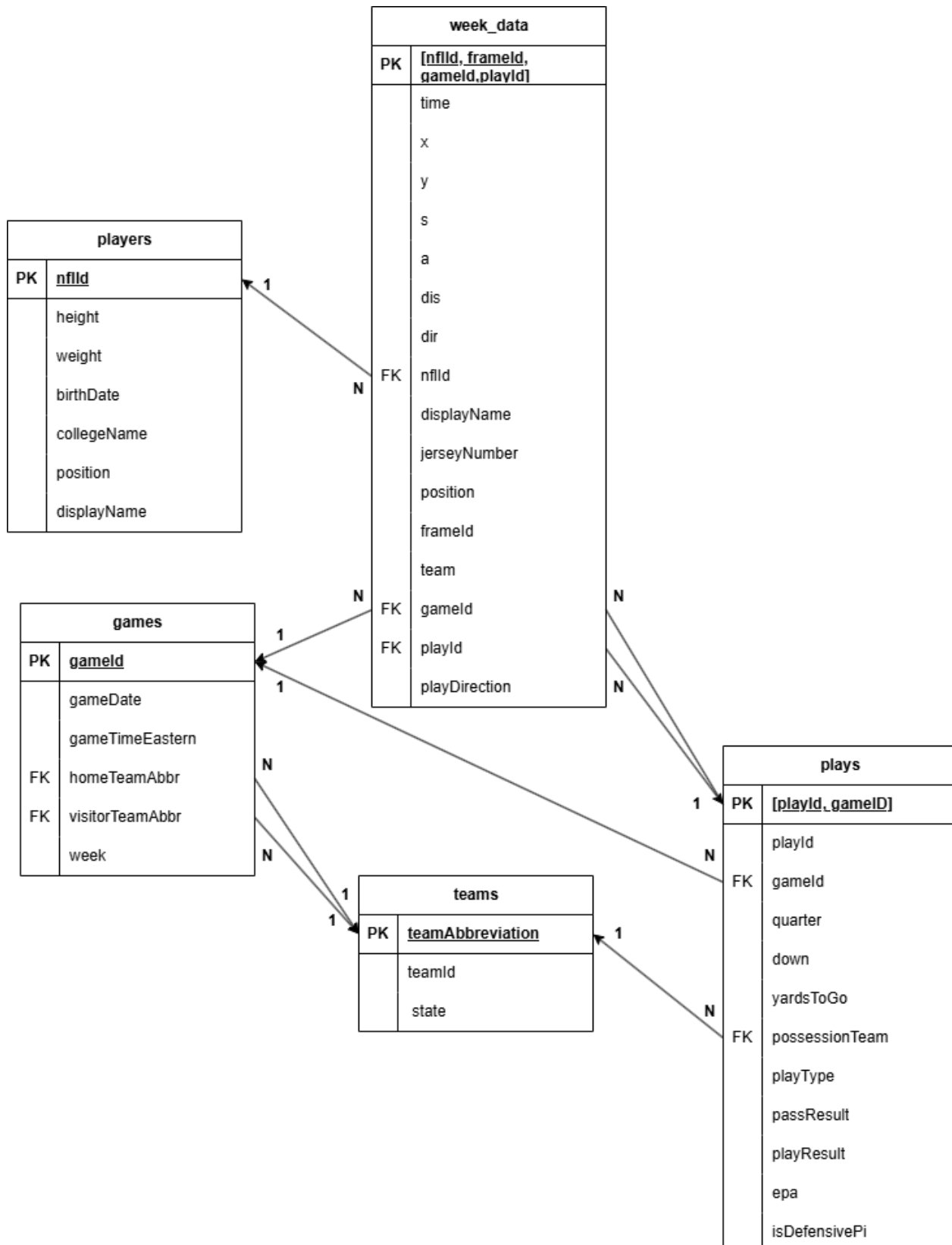
- Permite preservar la integridad de los datos, evitar redundancia y hacer joins naturales entre entidades relacionadas, como por ejemplo unir jugadas con partidos o movimientos de jugadores con sus datos personales

Como los datos refinados ya habían sido almacenados en las subcarpetas dentro de `/user/bigdata/zones/refined` no fue necesario crear nuevos archivos ni moverlos a otra zona de HDFS. Las tablas externas en Hive van a apuntar directamente a las carpetas dentro de `/refined/`, respetando la estructura.

Representación del modelo

Para representar el modelo hicimos un diagrama entidad-relación que muestra las claves primarias y las relaciones entre las cinco tablas del dataset.

Para hacerlo hicimos uso de draw.io



Tablas HIVE

En esta sección documentamos cómo fue el proceso de creación de tablas externas en HIVE. En nuestro caso, que elegimos un modelado normalizado, no fue necesario generar nuevos archivos adicionales, ya que como explicamos en la sección anterior decidimos utilizar este modelado porque ya estaba bien alineado con nuestros datos.

Las tablas que construimos en el refinamiento ya cuentan con la información limpia y estructurada.

Es decir:

- no tienen registros duplicados
- no tienen valores nulos
- los tipos de datos están ajustados
- las claves primarias fueron identificadas

Por lo tanto, las tablas externas de HIVE apuntarán directamente a las subcarpetas de /refined.

Organización previa al creado de tablas

Movimiento de datos crudos iniciales

Antes de crear las tablas comenzamos moviendo los datos crudos que teníamos en Ind a raw.

Como los datos ya habían sido tratados en /Ind y guardados en /refined (a donde apuntaran las tablas Hive) hicimos este movimiento dejando todos los datos crudos iniciales en la carpeta raw en forma de respaldo. De forma que si el día de mañana llega una tabla nueva a Ind para ser tratada y refinada, no se encuentren las tablas que ya pasaron por ese proceso.

En esta imagen vemos como hacemos el movimiento de Ind a raw con el primer comando. Luego con el segundo comando verificamos que los archivos ya no estén en Ind y con el último chequeamos que se hayan guardado correctamente en raw.

Verificación de archivos refinados

Luego de hacer esto, nos pareció bien corroborar que las tablas refinadas guardadas en /refined se hayan cargado como esperábamos, sin header y con los datos separados por “;”.

Este chequeo fue importante ya que las tablas externas en Hive van a apuntar acá para hacer las consultas utilizando Spark con sintaxis SQL.

En la imagen vemos como las tablas fueron cargadas de la forma esperada, por lo tanto estábamos listos para crear las tablas en Hive.

```
[ort@taller-bigdata-vm ~]$ hdfs dfs -cat /user/bigdata/zones/refined/teams/part-00000*.csv | head -5
28,SEA,Washington
17,LA,California
2,ATL,Georgia
32,WAS,Maryland
6,CHI,Illinois
[ort@taller-bigdata-vm ~]$ hdfs dfs -cat /user/bigdata/zones/refined/players/part-00000*.csv | head -5
2552647,69,195,1991-10-03,Missouri,RB,Marcus Murphy
2557980,71,213,1995-10-04,Michigan,FS,Jabrill Peppers
2558924,77,300,1994-04-12,Vanderbilt,DT,Adam Butler
2543465,74,215,1992-11-10,Louisville,QB,Teddy Bridgewater
2561359,72,180,1995-03-19,Texas Southern,WR,Darvin Kidsy
cat: Unable to write to output stream.
[ort@taller-bigdata-vm ~]$ hdfs dfs -cat /user/bigdata/zones/refined/games/part-00000*.csv | head -5
2018123015,2018-12-30,16:25:00,SEA,ARI,17
2018101100,2018-10-11,20:20:00,NYG,PHI,6
2018111107,2018-11-11,13:00:00,TB,WAS,10
2018112900,2018-11-29,20:20:00,DAL,NO,13
2018121610,2018-12-16,16:25:00,PIT,NE,15
[ort@taller-bigdata-vm ~]$ hdfs dfs -cat /user/bigdata/zones/refined/plays/part-00000*.csv | head -5
2018090902,3597,4,2,8,CIN,play_type_pass,C,18,1.60913697591926,false
2018090906,2897,3,2,8,NO,play_type_pass,I,0,-0.929154471339837,false
2018091001,524,1,1,10,LA,play_type_pass,I,0,-0.478412734869797,false
2018091001,2350,3,3,3,LA,play_type_pass,C,11,1.41942144969176,false
2018091605,823,1,2,5,PIT,play_type_pass,I,0,-0.558917920385846,false
cat: Unable to write to output stream.

[ort@taller-bigdata-vm ~]$ hdfs dfs -cat /user/bigdata/zones/refined/week_data/part-00000*.csv | head -5
time,x,y,s,a,dis,dir,nflId,displayName,jerseyNumber,position,frameId,team,gameId,playId,playDirection
2018-11-16T01:24:16.799Z,79.11,15.11,0,0,0,137.25,2560883,Equanimeous St. Brown,19,WR,11,away,2018111500,90,right
2018-11-16T01:30:10.500Z,84.08,34.56,0.02,0.56,0.01,292.85,497224,Ed Dickson,84,TE,13,home,2018111500,221,left
2018-11-16T01:30:12.599Z,72.63,18.11,3.48,1.61,0.34,289.05,2558119,Josh Jones,27,SS,34,away,2018111500,221,left
2018-11-16T01:30:13.299Z,80.77,39.8,5.35,1.35,0.53,325.08,2555340,Kyler Fackrell,51,OLB,41,away,2018111500,221,left
cat: Unable to write to output stream.
```

Creación de tablas externas en HIVE

Comenzamos ingresando a hive desde la terminal de la máquina virtual. Hecho esto, lo primero que hicimos fue crear una base de datos llamada nfl_data, que será el esquema donde van a estar todas las tablas que representan nuestro modelo normalizado.

En la imagen vemos los comandos ejecutados para hacer esto

```
hive> CREATE DATABASE IF NOT EXISTS nfl_data;
OK
Time taken: 0.917 seconds
hive> USE nfl_data;
OK
Time taken: 0.071 seconds
```

El primer comando asegura que la base de datos solo se cree si aún no existe, evitando errores si volvemos a correr el script en un futuro. Una vez que la creamos, establecimos el contexto de trabajo para usar la base de datos creada.

Ahora si comenzamos a crear las tablas externas en HIVE correspondientes a cada una de las entidades de nuestro modelo.

Para cada creación de tabla seguimos el mismo procedimiento:

- 1) Definimos la estructura de columnas en función del esquema limpio generado en el refinamiento
- 2) Especificamos el formato de los datos como TEXTFILE, con delimitación por “,”.
- 3) Indicamos que la tabla es externa, ya que hacen referencia a los archivos ubicados en HDFS en /refined, sin administrarlos de forma directa.
- 4) Establecimos la ruta exacta dentro de la zona /refined donde se encuentra el archivo correspondiente de cada tabla. Por ejemplo en el caso de la tabla teams la location fue /user/bigdata/zones/refined/teams

Cada vez que creamos una tabla, hicimos un SELECT * FROM (la tabla que fuera) LIMIT 5, esto nos permitía ver los primeros cinco registros de la tabla creada, pudiendo confirmar que se había creado correctamente.

En las siguientes imágenes vemos visualmente como fue realizado todo lo explicado:

Creación y verificación de registros de tabla teams:

```
hive> CREATE EXTERNAL TABLE teams (  
  > teamId INT,  
  > teamAbbreviation STRING,  
  > state STRING  
  > )  
  > ROW FORMAT DELIMITED  
  > FIELDS TERMINATED BY ','  
  > STORED AS TEXTFILE  
  > LOCATION '/user/bigdata/zones/refined/teams/';  
OK  
Time taken: 0.563 seconds  
hive> SELECT * FROM teams LIMIT 5;  
OK  
28      SEA      Washington  
17      LA       California  
2       ATL      Georgia  
32      WAS      Maryland  
6       CHI      Illinois  
Time taken: 1.966 seconds, Fetched: 5 row(s)
```

Creación y verificación de registros de tabla players:

```
hive> CREATE EXTERNAL TABLE players (  
  > nflId INT,  
  > height INT,  
  > weight INT,  
  > birthDate DATE,  
  > collegeName STRING,  
  > position STRING,  
  > displayName STRING  
  > )  
  > ROW FORMAT DELIMITED  
  > FIELDS TERMINATED BY ','  
  > STORED AS TEXTFILE  
  > LOCATION '/user/bigdata/zones/refined/players/';  
OK  
Time taken: 0.166 seconds  
hive> SELECT * FROM players LIMIT 5;  
OK  
2552647 69      195      1991-10-03      Missouri      RB      Marcus Murphy  
2557980 71      213      1995-10-04      Michigan      FS      Jabrill Peppers  
2558924 77      300      1994-04-12      Vanderbilt    DT      Adam Butler  
2543465 74      215      1992-11-10      Louisville    QB      Teddy Bridgewater  
2561359 72      180      1995-03-19      Texas Southern WR      Darvin Kidsy  
Time taken: 0.252 seconds, Fetched: 5 row(s)
```

Creación y verificación de registros de tabla games:

```
Time taken: 0.252 seconds, Fetched: 5 row(s)  
hive> CREATE EXTERNAL TABLE games (  
  > gameId INT,  
  > gameDate DATE,  
  > gameTimeEastern STRING,  
  > homeTeamAbbr STRING,  
  > visitorTeamAbbr STRING,  
  > week INT  
  > )  
  > ROW FORMAT DELIMITED  
  > FIELDS TERMINATED BY ','  
  > STORED AS TEXTFILE  
  > LOCATION '/user/bigdata/zones/refined/games/';  
OK  
Time taken: 0.143 seconds  
hive> SELECT * FROM games LIMIT 5;  
OK  
2018123015      2018-12-30      16:25:00      SEA      ARI      17  
2018101100      2018-10-11      20:20:00      NYG      PHI      6  
2018111107      2018-11-11      13:00:00      TB      WAS      10  
2018112900      2018-11-29      20:20:00      DAL      NO      13  
2018121610      2018-12-16      16:25:00      PIT      NE      15  
Time taken: 0.272 seconds, Fetched: 5 row(s)
```

Creación y verificación de registros de tabla plays:

```
Time taken: 0.272 seconds, Fetched: 5 row(s)
hive> CREATE EXTERNAL TABLE plays (
  > gameId INT,
  > playId INT,
  > quarter INT,
  > down INT,
  > yardsToGo INT,
  > possessionTeam STRING,
  > playType STRING,
  > passResult STRING,
  > playResult INT,
  > epa DOUBLE,
  > isDefensivePI BOOLEAN
  > )
  > ROW FORMAT DELIMITED
  > FIELDS TERMINATED BY ','
  > STORED AS TEXTFILE
  > LOCATION '/user/bigdata/zones/refined/plays/';
OK
Time taken: 0.138 seconds
hive> SELECT * FROM plays LIMIT 5;
OK
2018090902      3597    4      2      8      CIN      play_type_pass C      18      1.60913697591926      false
2018090906      2897    3      2      8      NO      play_type_pass I      0      -0.929154471339837      false
2018091001      524     1      1     10      LA      play_type_pass I      0      -0.478412734869797      false
2018091001      2350    3      3      3      LA      play_type_pass C     11      1.41942144969176      false
2018091605      823     1      2      5      PIT      play_type_pass I      0      -0.558917920385846      false
Time taken: 0.313 seconds, Fetched: 5 row(s)
```

Creación y verificación de registros de tabla week_data:

```
hive> CREATE EXTERNAL TABLE week_data (
  > eventTime STRING,
  > x STRING,
  > y STRING,
  > s STRING,
  > a STRING,
  > dis STRING,
  > dir STRING,
  > nflId INT,
  > displayName STRING,
  > jerseyNumber INT,
  > position STRING,
  > frameId INT,
  > team STRING,
  > gameId INT,
  > playId INT,
  > playDirection STRING
  > )
  > ROW FORMAT DELIMITED
  > FIELDS TERMINATED BY ','
  > STORED AS TEXTFILE
  > LOCATION '/user/bigdata/zones/refined/week_data/';
OK
Time taken: 0.115 seconds
hive> SELECT * FROM week_data LIMIT 5;
OK
2018-11-16T01:24:16.799Z      79.11    15.11    0      0      0      137.25    2560883 Equanimeous St. Brown    19    WR    11    away    2018111500    221    left    right
2018-11-16T01:30:10.500Z      84.08    34.56    0.02    0.56    0.01    292.85    497224 Ed Dickson      84    TE    13    home    2018111500    221    left
2018-11-16T01:30:12.599Z      72.63    18.11    3.48    1.61    0.34    289.05    2558119 Josh Jones      27    SS    34    away    2018111500    221    left
2018-11-16T01:30:13.299Z      80.77    39.8     5.35    1.35    0.53    325.08    2555340 Kyler Fackrell    51    OLB   41    away    2018111500    221    left
2018-11-16T01:30:14.900Z      85.88    19.2     2.41    1.14    0.25    204.96    2555289 Nick Vannett     81    TE    57    home    2018111500    221    left
Time taken: 0.233 seconds, Fetched: 5 row(s)
```

Respuesta a las preguntas con HIVE y Spark

Luego de haber definido las tablas en HIVE, creamos un notebook nuevo en Jupyter llamado consultas_hive_nfl para poder responder las cinco preguntas elegidas.

Estas preguntas fueron validadas con los docentes y nos permiten explorar distintos aspectos del dataset, combinando la información de las tablas a través de sus claves.

Las preguntas que elegimos fueron:

- 1) ¿Cuáles son los estados con mayor cantidad de partidos disputados?
- 2) ¿Cuáles fueron los 5 equipos que generaron la mayor cantidad de jugadas ofensivas con impacto positivo en la probabilidad de anotar en cada semana de la temporada?
- 3) ¿Qué jugadores alcanzaron la mayor velocidad promedio por posición ofensiva en jugadas de pase exitoso?
- 4) ¿Qué equipos permitieron la menor cantidad de yardas por jugada en contra a lo largo de la temporada?
- 5) ¿Qué universidades formaron a los 10 jugadores que más distancia recorrieron durante toda la temporada?

Para poder responder las preguntas usamos sintaxis SQL sobre las tablas HIVE, ejecutadas mediante Spark SQL, evitando acceder a los archivos refinados mediante lectura directa a DataFrames.

Aclaración: en esta sección solo mencionamos de forma general lo que hicimos ya que el detalle completo de las consultas como el código SQL, la explicación, los resultados y las interpretaciones se encuentran dentro del notebook.

Guardado de respuestas

Una vez que respondimos las cinco preguntas en el notebook, guardamos los resultados de cada pregunta en la zona /analytics en HDFS.

Cada resultado se guardó en una subcarpeta específica para tener mayor organización, usando los distintos formatos de archivos con la intención de aplicar los formatos que vimos en el curso.

Las preguntas 1,2,4 y 5 fueron almacenadas en formato parquet por su eficiencia para trabajar con grandes volúmenes de datos. Como es un formato columnar, permite leer solo las columnas necesarias, haciendo que las consultas sean más rápidas. Además de esto tiene la ventaja de ocupar menos espacio.

En cuanto a la pregunta 3 decidimos usar formato CSV para mostrar el uso de otro formato de guardado. Luego de guardar la respuesta de esta pregunta en /analytics/respuesta_pregunta_3_csv, verificamos que la respuesta se haya guardado correctamente.

En la imagen vemos como el resultado guardado es el mismo que el del notebook.

```
[ort@taller-bigdata-vm ~]$ hdfs dfs -cat /user/bigdata/zones/analytics/respuesta_pregunta_3_csv/part-*.csv
nflId,position,avg_speed
2559270,TE,6.771478260869564
2505996,QB,2.035914332784184
2557952,RB,4.5708552631578945
2550235,FB,4.976315789473685
2552656,WR,5.483678756476683
```

nflId	position	avg_speed
2559270	TE	6.771478260869564
2505996	QB	2.035914332784184
2557952	RB	4.5708552631578945
2550235	FB	4.976315789473685
2552656	WR	5.483678756476683

En las siguiente imagen dejamos prueba de que el resto de respuestas fueron guardadas correctamente en la zona esperada.

```
[ort@taller-bigdata-vm ~]$ hdfs dfs -ls /user/bigdata/zones/analytics/respuesta_pregunta_1
Found 2 items
-rw-r--r-- 1 ort supergroup 0 2025-07-01 00:01 /user/bigdata/zones/analytics/respuesta_pregunta_1/_SUCCESS
-rw-r--r-- 1 ort supergroup 1055 2025-07-01 00:01 /user/bigdata/zones/analytics/respuesta_pregunta_1/part-00000-cc3e5689-bbf1-4177-aeaa-eac09fe4b788-c000.snappy.parquet
[ort@taller-bigdata-vm ~]$ hdfs dfs -ls /user/bigdata/zones/analytics/respuesta_pregunta_2
Found 2 items
-rw-r--r-- 1 ort supergroup 0 2025-07-01 00:06 /user/bigdata/zones/analytics/respuesta_pregunta_2/_SUCCESS
-rw-r--r-- 1 ort supergroup 2060 2025-07-01 00:06 /user/bigdata/zones/analytics/respuesta_pregunta_2/part-00000-b5f1dea1-2e2a-4b52-bc8a-5180a98ad2e7-c000.snappy.parquet
[ort@taller-bigdata-vm ~]$ hdfs dfs -ls /user/bigdata/zones/analytics/respuesta_pregunta_4
Found 2 items
-rw-r--r-- 1 ort supergroup 0 2025-07-01 00:09 /user/bigdata/zones/analytics/respuesta_pregunta_4/_SUCCESS
-rw-r--r-- 1 ort supergroup 1705 2025-07-01 00:09 /user/bigdata/zones/analytics/respuesta_pregunta_4/part-00000-b130d107-d64a-4c72-ad69-fcc2bc310453-c000.snappy.parquet
[ort@taller-bigdata-vm ~]$ hdfs dfs -ls /user/bigdata/zones/analytics/respuesta_pregunta_5
Found 2 items
-rw-r--r-- 1 ort supergroup 0 2025-07-01 00:11 /user/bigdata/zones/analytics/respuesta_pregunta_5/_SUCCESS
-rw-r--r-- 1 ort supergroup 869 2025-07-01 00:11 /user/bigdata/zones/analytics/respuesta_pregunta_5/part-00000-2e0c8dbf-a8c2-44a3-a282-324f0ba9a7f2-c000.snappy.parquet
```

Visualización de Resultados en Notebook

Luego de guardar los resultados de las cinco preguntas, seleccionamos las preguntas 2,3 y 5 para hacer distintos tipos de visualizaciones en el notebook.

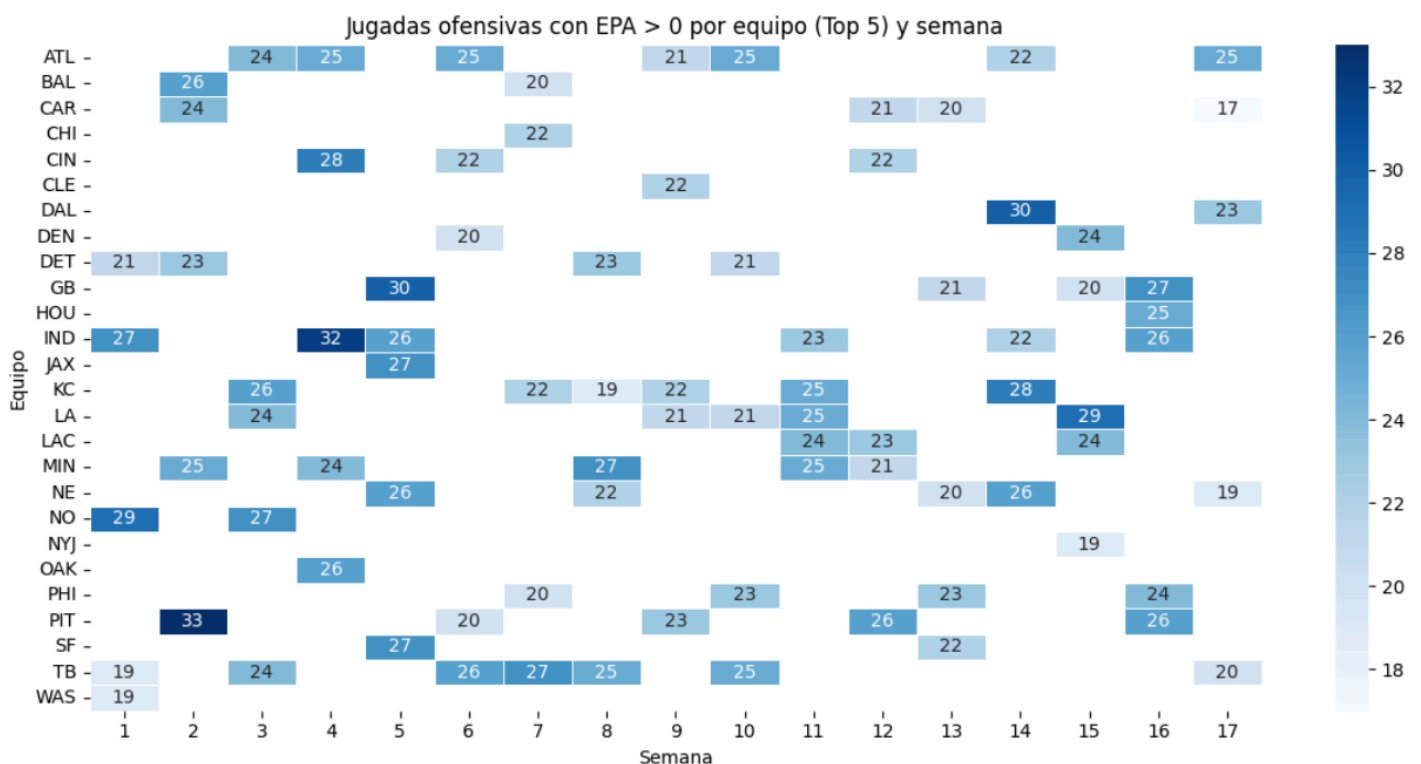
En nuestro caso fue necesario instalar algunas librerías para poder graficar, estas fueron: matplotlib, pandas y seaborn.

Una vez instaladas empezamos a hacer la visualización de cada pregunta.

Pregunta 2: ¿Cuáles fueron los 5 equipos que generaron la mayor cantidad de jugadas ofensivas con impacto positivo en la probabilidad de anotar en cada semana?

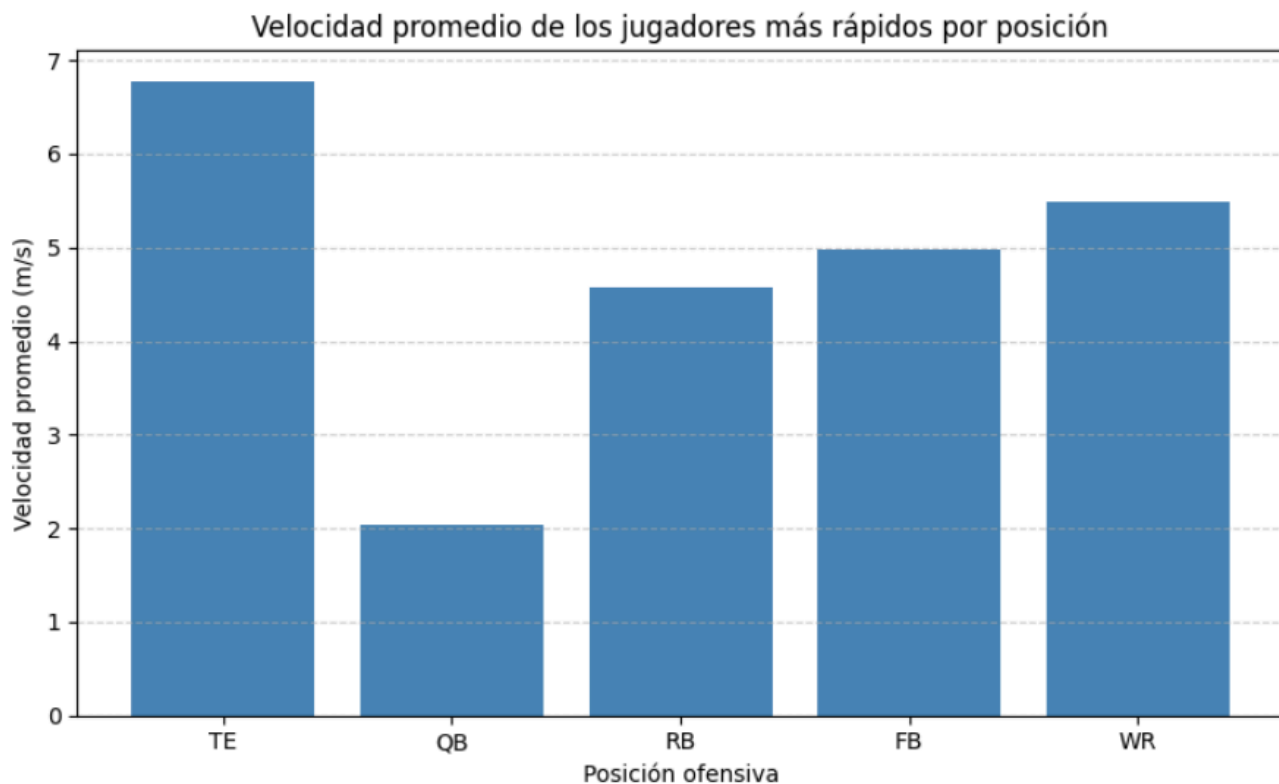
Para visualizar la respuesta de esta pregunta optamos por un mapa de calor en forma de matriz, con equipos en el eje Y, semanas en el eje X y valores de jugadas con EPA > 0.

El motivo por el que elegimos esta forma de visualización es porque el resultado tiene estructura matricial. Para esto usamos la librería seaborn y el resultado obtenido fue:



Pregunta 3: ¿Qué jugadores alcanzaron la mayor velocidad promedio por posición ofensiva en jugadas de pase exitoso?

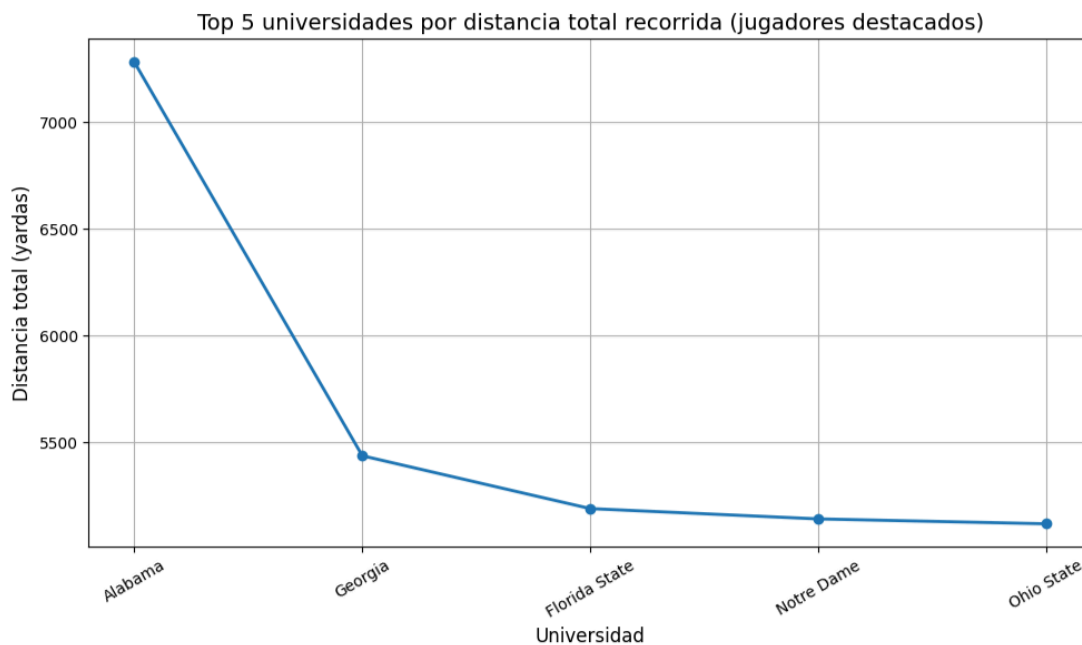
Para esta pregunta elegimos una visualización de gráfico de barras por posición ofensiva, el resultado es un valor promedio por categoría, que se puede representar de forma clara usando barras. Para esta visualización usamos la librería matplotlib y obtuvimos:



Pregunta 5: ¿Qué universidades formaron a los 10 jugadores que más distancia recorrieron durante toda la temporada?

Para la visualización de esta pregunta elegimos un gráfico de líneas con puntos marcados. El eje X representa las universidades que conforman el top 5, mientras que el eje Y la distancia total recorrida por sus jugadores.

Esta representación resalta bien la diferencia de distancias recorridas entre universidades, principalmente la diferencia entre Alabama y el resto. Además la elegimos para dar variedad en cuanto a la forma de visualización.



Visualización de resultados en Superset

Para realizar la visualización en superset elegimos las preguntas 1, 4 y 5. Para hacer esto comenzamos creando nuevas tablas externas en HIVE, esta vez apuntando a los resultados de las preguntas. Estos resultados se obtuvieron realizando consultas SQL en el segundo notebook y fueron guardadas en nuestra zona /analytics., por lo tanto las nuevas tablas que creamos apuntan a dicha zona.

En las siguientes imágenes vemos cómo creamos las tres tablas (todas fueron creadas dentro de la base de datos nfl_data creada anteriormente

Creación de nuevas tablas en Hive

Creación de tablas para pregunta 1:

```
hive> USE nfl_data;
OK
Time taken: 0.045 seconds
hive> CREATE EXTERNAL TABLE IF NOT EXISTS respuesta_pregunta_1_iso (
  >   state_iso STRING,
  >   partidos INT
  > )
  > STORED AS PARQUET
  > LOCATION '/user/bigdata/zones/analytics/respuesta_pregunta_1_iso';
OK
Time taken: 0.364 seconds
hive> SELECT * FROM respuesta_pregunta_1_iso LIMIT 5;
OK
US-CA    62
US-FL    48
US-OH    32
US-TX    32
US-PA    32
Time taken: 1.448 seconds, Fetched: 5 row(s)
```

Creación de tablas para pregunta 4:

```
hive> CREATE EXTERNAL TABLE IF NOT EXISTS respuesta_pregunta_4 (
  >   state STRING,
  >   equipo STRING,
  >   avg_yards_allowed DOUBLE
  > )
  > STORED AS PARQUET
  > LOCATION '/user/bigdata/zones/analytics/respuesta_pregunta_4';
OK
Time taken: 0.629 seconds
hive> SELECT * FROM respuesta_pregunta_4 LIMIT 5;
OK
Maryland      BAL      5.3
Illinois      CHI      5.34
Minnesota     MIN      5.56
New York      BUF      5.63
Arizona ARI    5.85
Time taken: 1.791 seconds, Fetched: 5 row(s)
```

Creación de tablas para pregunta 5:

```
hive> CREATE EXTERNAL TABLE IF NOT EXISTS respuesta_pregunta_5 (  
  >   collegeName STRING,  
  >   suma_top_10 DOUBLE  
  > )  
  > STORED AS PARQUET  
  > LOCATION '/user/bigdata/zones/analytics/respuesta_pregunta_5';  
OK  
Time taken: 0.136 seconds  
hive> SELECT * FROM respuesta_pregunta_5 LIMIT 5;  
OK  
Alabama 7281.3199999999915  
Georgia 5436.5599999999992  
Florida State 5188.9899999999993  
Notre Dame 5140.6899999999996  
Ohio State 5117.8699999999995  
Time taken: 0.218 seconds, Fetched: 5 row(s)
```

Aclaración: hicimos tres visualizaciones ya que al final nos dimos cuenta que la pregunta 1 podría representarse de forma más enriquecedora en superset y ya habíamos hecho dos visualizaciones. A su vez la pregunta 5 fue representada en superset como un pie chart, esta visualización no es muy clara para la respuesta que da la pregunta, por lo tanto también la representamos en Spark como gráfico con puntos.

Ingresamos a Superset

Luego de crear las tablas instalamos Superset en nuestra consola e ingresamos a través de <http://superset.bigdata.ort.edu.uy> con el usuario ort.

Una vez dentro creamos un dashboard único al que llamamos Visualización Superset y contiene las visualizaciones de las tres preguntas.

En este caso nos pareció apropiado crear un único dashboard que es el que vamos a entregar, sin embargo podríamos haber creado un dashboard por pregunta, de esta forma si más adelante queremos hacer nuevas visualizaciones las íbamos a poder guardar de forma organizada y separada.

Creación de Charts

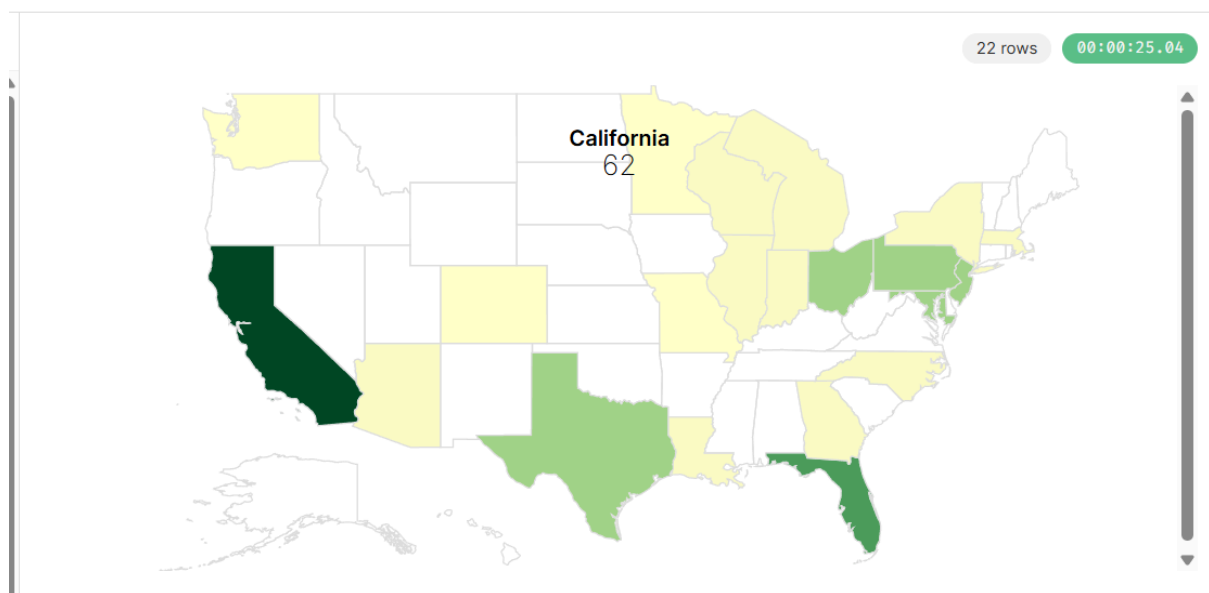
Aquí comenzamos a crear los charts correspondientes a cada visualización, estos luego se guardaron en el dashboard. Para cada chart lo primero que hicimos fue elegir la base de datos configurada en Hive (nfl_data), indicando que tabla Hive de la base de datos íbamos a utilizar para la visualización.

Pregunta 1: ¿Cuáles son los estados con mayor cantidad de partidos disputados?

Para visualizar la respuesta de la pregunta 1 creamos un chart de tipo Country Map, este es un mapa de calor que nos permite elegir el país en donde se van a desplegar los datos. En nuestro caso el país es Estados Unidos.

Por lo tanto al configurarlo aclaramos Country = USA, luego en la opción ISO Codes cargamos nuestra columna state_iso, esta columna tiene todos los estados abreviados en formato ISO para que el chart pueda reconocerlos y pintarlos correctamente. Por último pusimos como métrica SUM(partidos).

El resultado obtenido fue:



Como vemos, cuanto más intenso el verde, más partidos se jugaron. En nuestro caso el estado donde más partidos se jugaron fue California. Cabe aclarar que la NFL disputa partidos solo en 22 estados de Estados Unidos, es por eso que muchos de ellos están sin colorear.

Configuración del chart:

DATA

☒ COUNTRY MAP ☐ ☐ ☐ ☐ 4k ☐

[View all charts](#)

Time

Query

COUNTRY

USA

ISO 3166-2 CODES

abc state_iso

METRIC

f(x) SUM(partidos)

FILTERS

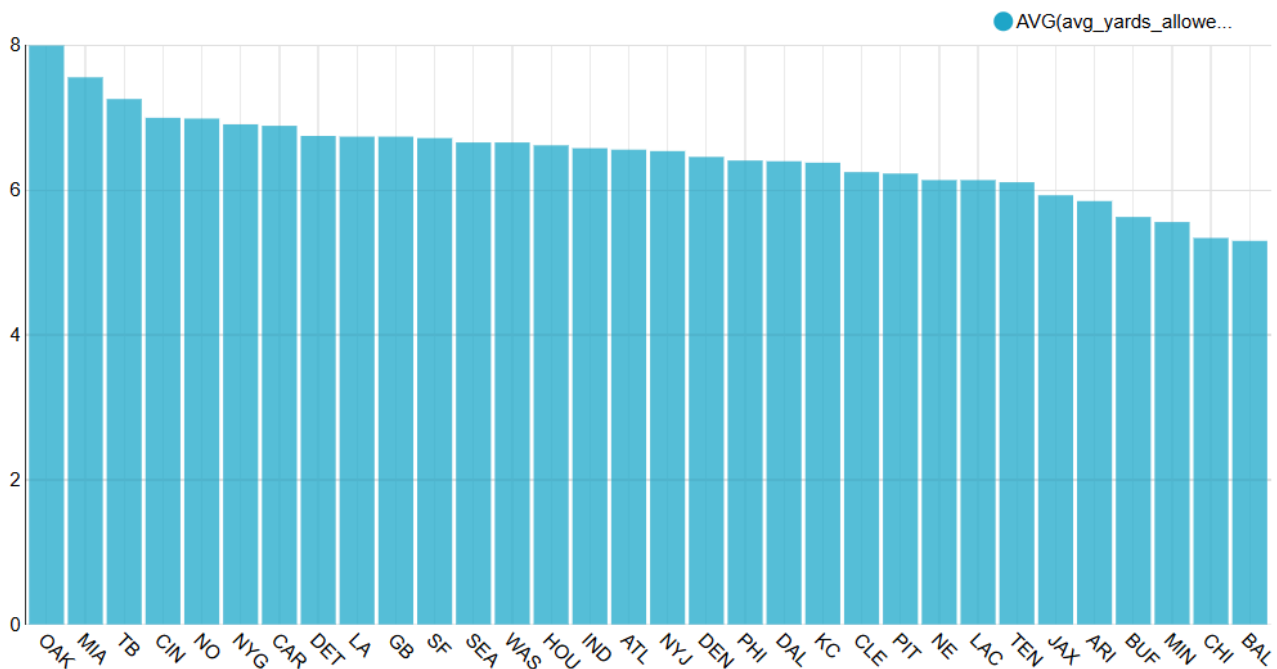
Drop columns/metrics here or click

Chart Options

Pregunta 4: ¿Qué equipos permitieron la menor cantidad de yardas por jugada en contra a lo largo de la temporada?

Para esta pregunta optamos por un Bar Chart, este tipo de visualización también la tenemos en el notebook pero nos pareció interesante y una forma clara de mostrar los resultados en Superset.

En el eje X configuramos a los equipos y en el eje Y la cantidad de yardas que cedieron o permitieron avanzar en jugadas defensivas a lo largo de la temporada.



Configuración aplicada:

DATA

CUSTOMIZE

☒ BAR CHART ☐ LINE ☐ AREA ☐ TABLE ☐ 4k ☐

[View all charts](#)

Time

Query

METRICS

f(x) AVG(avg_yards_allowed)

Drop columns/metrics here or click

FILTERS

Drop columns/metrics here or click

DIMENSIONS

abc equipo

Drop columns here or click

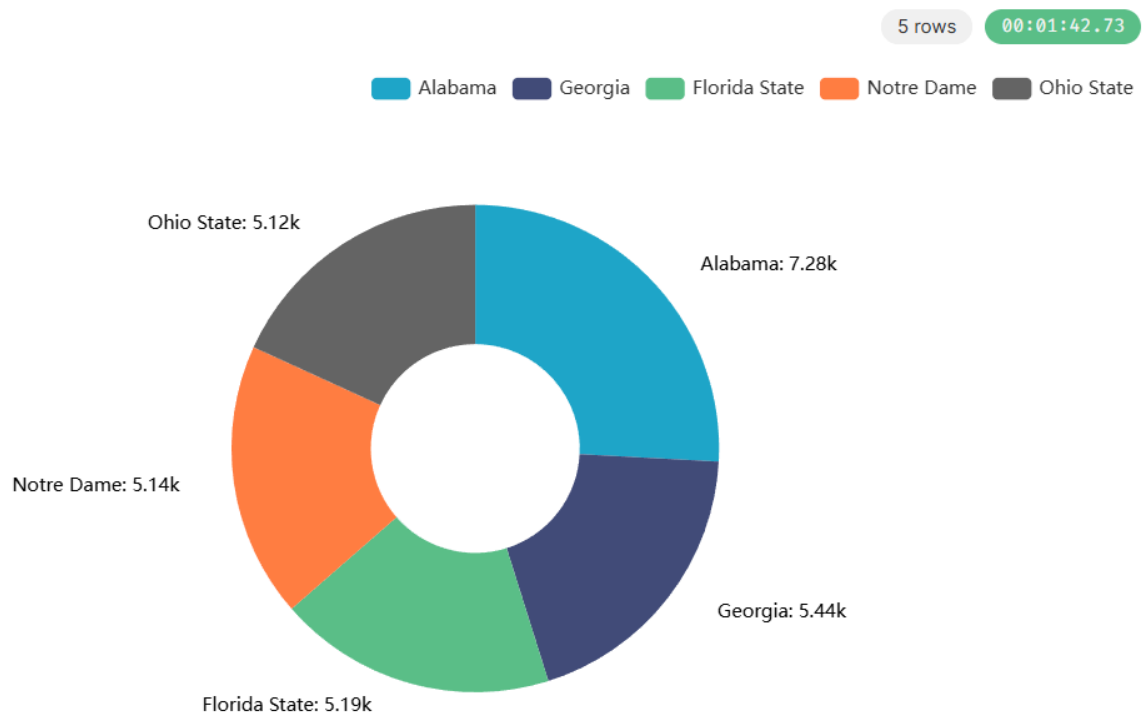
BREAKDOWNS

Drop columns here or click

Pregunta 5: ¿Qué universidades formaron a los 10 jugadores que más distancia recorrieron durante toda la temporada?

Finalmente para visualizar esta respuesta elegimos un Pie Chart, en un principio creímos que era buena opción para dar variedad y probar los distintos charts que ofrece Superset. Sin embargo luego de hacerlo nos dimos cuenta que no era claro ya que los resultados de las distancias recorridas por universidad eran todos bastante similares. Es por esto que nos pareció correcto representarlo también en el notebook aunque tuviéramos que hacer una visualización extra.

El resultado obtenido fue:



Configuración aplicada:

DATA CUSTOMIZE

4k PIE CHART

[View all charts](#)

Time

Query

DIMENSIONS

abc collegename

+ Drop columns here or click

METRIC

f(x) SUM(suma_top_10)

FILTERS

+ Drop columns/metrics here or click

ROW LIMIT

10

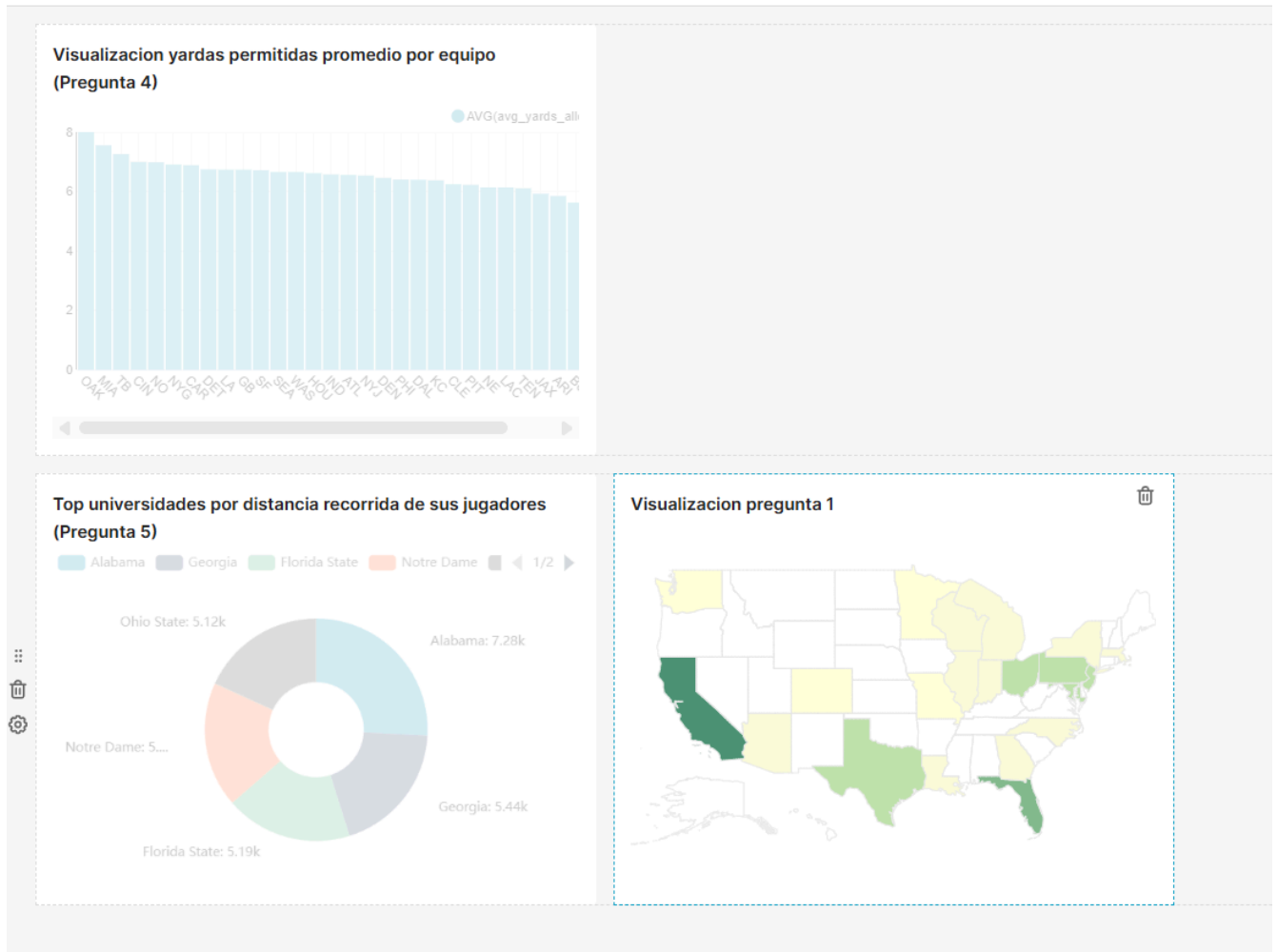
☒ SORT BY METRIC

UPDATE CHART

Dashboard Final

En esta sección dejamos una imagen en forma de evidencia de cómo quedó finalmente nuestro dashboard, conteniendo los tres charts realizados.

Visualizacion Superset



Parte 2 - Arquitectura Alternativa para un Data Lake

En esta segunda parte del trabajo proponemos una arquitectura alternativa a la de la primera parte para la construcción de un data lake, buscando dar uso a herramientas distintas, pero manteniendo Spark como lo pide la letra, siendo el motor del procesamiento distribuido.

Esta nueva solución está orientada a ser modular, flexible y que se pueda implementar tanto en entornos open source como híbridos, permitiendo su despliegue en infraestructura local y en la nube.

Stack tecnológico del nuevo data lake

Para esta arquitectura alternativa seleccionamos seis tecnologías principales, cada una cubriendo una etapa específica del ciclo de vida de los datos dentro del data lake.

1) Apache Flume

Es una herramienta pensada para recolectar y transportar grandes volúmenes de datos desde distintas fuentes. En nuestro caso, lo elegimos porque permite configurar de forma simple flujos de datos desde archivos, logs o servicios externos y redirigirlos automáticamente a un destino predefinido. Es ideal para entornos donde se requiere ingesta continua o periódica sin necesidad de lógica personalizada.

2)Ceph

Ceph es el sistema de almacenamiento distribuido elegido para guardar los datos del data lake. Usamos su componente RADOS Gateway, que permite almacenar archivos de forma escalable y compatible con la API de S3, sin depender de servicios en la nube. Esto facilita la integración con otras herramientas y nos da mayor control sobre los datos.

3)Apache Spark

Utilizando su interfaz PySpark, se encarga del procesamiento distribuido de los datos. Nos permite aplicar transformaciones complejas, limpiezas, enriquecimientos y cálculos analíticos sobre grandes volúmenes de datos. Su integración con sistemas de almacenamiento como Ceph y con motores de catálogo como Iceberg lo convierte en el núcleo del pipeline de datos.

4)Apache Iceberg

Es un motor de gestión de tablas diseñado para ambientes de big data. Nos permite definir estructuras de datos sobre archivos almacenados (como los generados por Spark), añadiendo capacidades avanzadas como transacciones ACID, versionado, evolución de

esquemas y lectura incremental. Esto nos garantiza trazabilidad y consistencia en todo el proceso analítico, reemplazando a herramientas más antiguas como Hive.

5)Trino

Es un motor de consulta distribuido que permite ejecutar SQL sobre múltiples fuentes de datos, incluyendo Iceberg, con alta performance. Es el componente que conecta Iceberg con Redash, facilitando la exploración de grandes volúmenes de datos estructurados de forma rápida y escalable.

6)Redash

Es la herramienta elegida para la visualización de los datos. Está orientada a usuarios técnicos y permite conectarse a múltiples fuentes mediante SQL para explorar los datos y construir dashboards interactivos. Su simplicidad de uso y rápida integración la hacen ideal para equipos de análisis que buscan flexibilidad sin depender de herramientas más pesadas o complejas.

Flujo del proceso

El flujo de datos empieza en los sistemas de origen, ya sea una API, archivos CSV, logs de servidores o datos transmitidos desde sensores IoT.

Estos datos son recolectados por Apache Flume, que actúa como puente entre la fuente de datos y el almacenamiento distribuido, como lo hizo NiFi para la primera parte.

Flume detecta datos nuevos automáticamente, los encapsula en eventos y los envía al destino configurado, en este flujo el destino es el bucket correspondiente en Ceph, dentro de la zona landing.

Una vez que se almacenaron los datos crudos, Spark accede a los archivos de la zona landing, permitiendo hacer los ajustes y refinamientos necesarios. Como se mencionó anteriormente luego de esto se guardan en refined.

Sobre estos archivos procesados, se crean tablas gestionadas por Apache Iceberg. Este paso agrega una capa de control transaccional que permite realizar operaciones de actualización, inserción y eliminación de datos sin perder consistencia. Además permite realizar consultas incrementales, como obtener solo los datos nuevos desde la última ejecución.

Finalmente, para explorar y visualizar los datos conectamos Redash a las tablas Iceberg usando Trino como motor de consultas intermedio. Esta integración permite a los usuarios

hacer consultas SQL, explorando los datos, construyendo dashboards, y compartiendo resultados.

Hecho todo esto se da por cerrado el ciclo desde la captura de datos hasta su visualización, haciendo uso de la nueva arquitectura que elegimos.

Comparación entre arquitecturas

Nos pareció interesante luego de plantear la arquitectura alternativa, hacer una pequeña comparación entre la arquitectura original del obligatorio y está, pudiendo ver mejor a las tecnologías “sustitutas” y en qué etapa actúan de forma equivalente a las tecnologías de la arquitectura original.

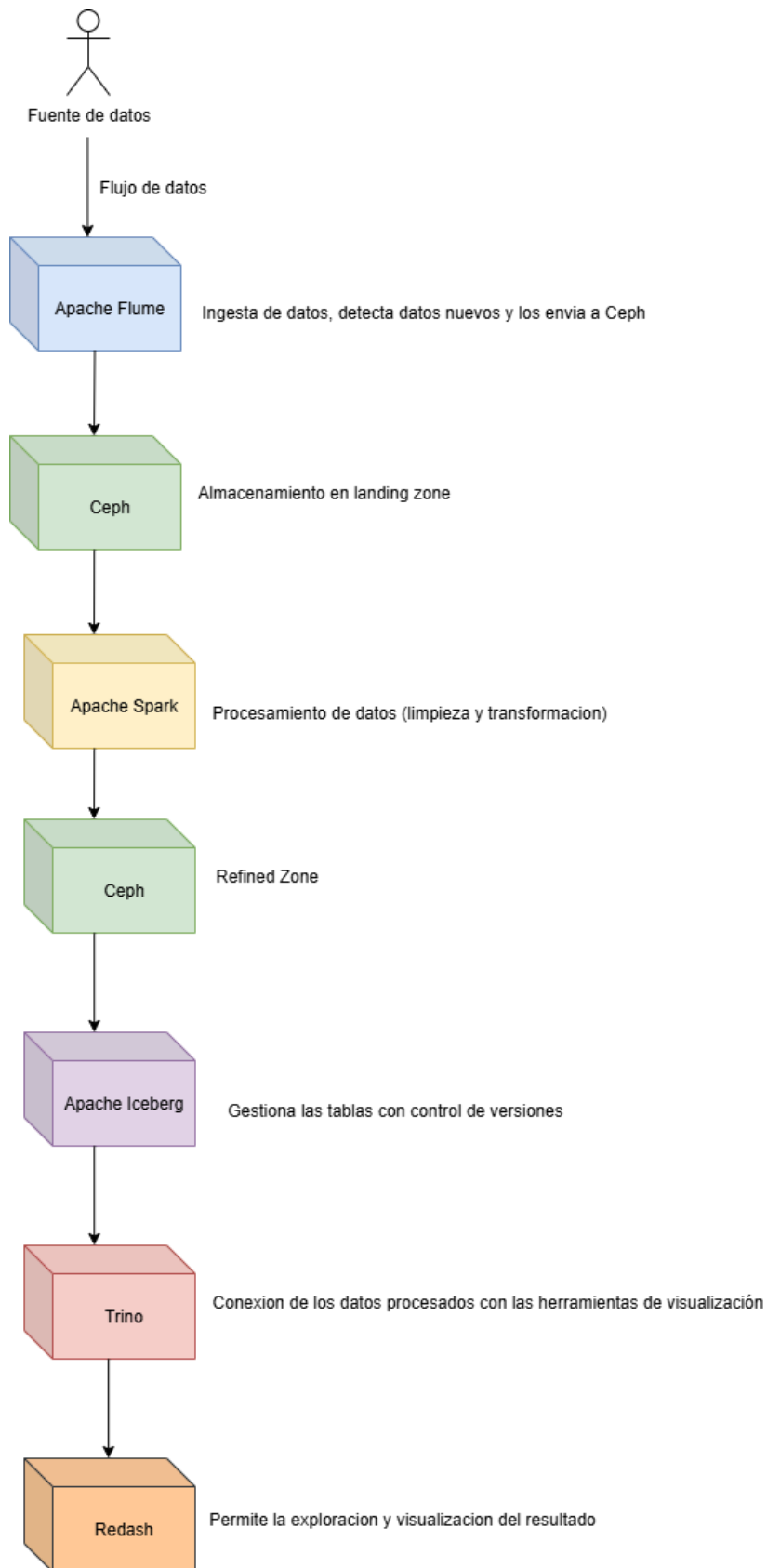
Etapas del pipeline	Arquitectura original	Arquitectura alternativa
Ingesta de datos	Apache NiFi	Apache Flume
Almacenamiento	HDFS (zonas landing, raw, refined, analytics)	Ceph (compatible con S3, con zonas equivalentes)
Procesamiento	Apache Spark (PySpark)	Apache Spark (PySpark)
Modelo de datos / catálogo	Hive (tablas externas)	Apache Iceberg (tablas gestionadas con ACID, versionado)
Motor de consultas	Hive (consultas desde notebooks o Superset)	Trino (consultas distribuidas y conexión con Redash)
Visualización	Superset + Jupyter Notebooks	Redash

Diferencias clave:

- En la ingesta, NiFi ofrece una interfaz más visual y flexible, mientras que Flume es más simple y enfocado a flujos continuos.
- El uso de Ceph permite simular un entorno cloud sin costo, mientras que HDFS es más clásico en arquitecturas on-premise.
- Iceberg ofrece ventajas sobre Hive como versionado, control de esquemas y actualizaciones seguras.
- Trino permite consultar múltiples fuentes con alto rendimiento, a diferencia de Hive que depende más del almacenamiento subyacente.
- En visualización, Superset es más completo pero requiere más configuración. Redash es más ágil y liviano para usuarios técnicos.

La arquitectura original nos permitió aplicar todos los conocimientos vistos en el curso, por otro lado la arquitectura alternativa nos dio una visión más moderna y flexible de Big Data actual.

Visualización del flujo de datos



Organización por zonas en el data lake

Para mantener las buenas prácticas de esta nueva arquitectura, definimos distintas zonas lógicas dentro de Ceph.

Estas zonas van a representar distintos estados en el ciclo de vida de los datos, dando orden, trazabilidad y control de calidad para cada etapa del flujo.

- 1) **Landing Zone:** es la zona de entrada del data lake. Aquí se almacenan los datos tal como llegan desde los sistemas de origen. Esta zona actúa como respaldo de los datos crudos, permitiendo auditar la información, hacer reprocesamientos si es necesario y conservar una copia sin alteraciones. Flume escribe directamente en esta zona.
- 2) **Refined Zone:** en esta etapa se almacenan los datos ya procesados por Spark. Los datos ya están validados, limpiados, transformados y analizados. Esta zona es la base para el modelado analítico y para la creación de las tablas con Iceberg
- 3) **Curated Zone:** esta tercera zona se puede crear para almacenar productos analíticos finales, como pueden ser métricas agregadas, reportes preprocesados y más. Su contenido está listo para ser consumido por Redash. Es útil para optimizar el rendimiento y simplificar el acceso a datos que se usan con frecuencia, sin necesidad de reprocesar el contenido de refined.
De todas formas esta zona es opcional ya que no todos los pipeline analíticos requieren una capa de datos finales específicos y se puede consultar directamente en refined.

Conclusiones Finales

Como cierre del trabajo nos pareció bueno dedicar una sección para hacer un repaso de lo que nos dejó el obligatorio.

Creemos que la propuesta nos sirvió para entender cómo trabajan de forma unificada las herramientas que vimos en el curso, ya que al hacer los prácticos específicos de cada tema, no había tanta integración entre ellos.

Por lo tanto pensamos que lo mejor del obligatorio, fue poder obtener un aprendizaje más global de todo el proceso que viven los datos para poder analizarlos. Desde el momento de su llegada, tratarlos de la mejor forma posible para poder analizarlos bien y por último poder visualizar los resultados.

Por último la parte 2 nos sirvió para ver aún más herramientas de trabajo que no se habían visto en el curso, permitiéndonos conocer aún más sobre las herramientas de Big Data.